Hardware Store

Java Desktop Application

Riccardo Petrone & Giuseppe Romanucci · Academic Year 2025/2026

INTRODUCTION

Digital Inventory Management for Small Hardware Shops

Hardware Store is a Java desktop application designed for small hardware shop owners and store managers who need a straightforward tool to track products — including name, category, brand, price, quantity, and supplier — without the complexity of enterprise software.

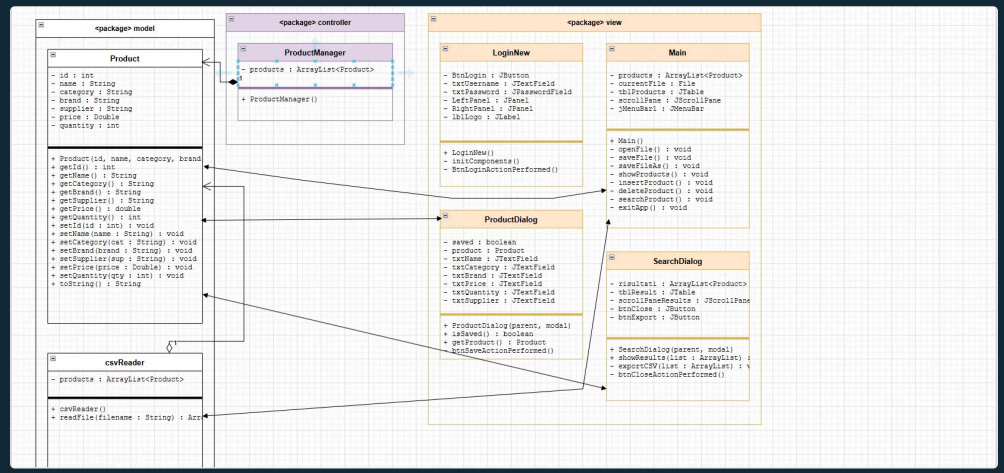
The application runs locally and stores all data in CSV files, ensuring simplicity, portability, and zero dependency on external systems.

ARCHITECTURE

Architecture Overview

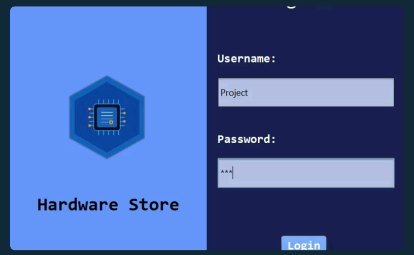
The application follows a clean **view-model separation**. All GUI components belong to `view` package, while data logic resides in `model`. The `Product` class serves as the shared data model across all layers.

- **LoginNew** → opens **Main**
- **Main** → uses **csvReader** and **Product**
- **Main** → opens **ProductDialog** and **SearchDialog**



GUI WALKTHROUGH

The 3 Main Screens

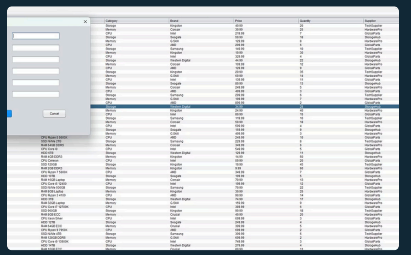


Login Screen

Two-panel layout: logo and app name on a light blue background (right), login form with username, password, and button on a dark navy panel (left).

Main Screen

Menu bar with File, Edit, and About menus. Central area displays the product table loaded from CSV.



Insert Dialog

Displays filtered results in a read-only table with an Export CSV button for saving search results.

FILE MANAGEMENT

CSV-Based Data Storage

The application uses **CSV (Comma-Separated Values)** as its native file format — simple, portable, and human-readable. The ID field is auto-incremented for every new product inserted.

File structure: ID, Name, Category, Brand, Price, Quantity, Supplier

- **Load:** File → Open via `BufferedReader` in `csvReader`
- **Save:** File → Save or SaveAs via `FileWriter`
- **Export:** Search results exported as a separate CSV from SearchDialog

CODE SNIPPET #1

CSV File Reading

```
public ArrayList<Product> readFile(String filename) {
  try (BufferedReader br =
       new BufferedReader(new FileReader(filename)))
{
    String line = null;
    String label = br.readLine(); // skip header
    while ((line = br.readLine()) != null) {
      String info[] = line.split(",");
      int id = Integer.parseInt(info[0]);
      String name = info[1];
      String category = info[2];
      String brand = info[3];
      Double price = Double.valueOf(info[4]);
      int quantity = Integer.parseInt(info[5]);
      String supplier = info[6];
      products.add(new Product(id, name,
        category, brand, price, quantity, supplier));
    }
  } catch (IOException e) {
    System.out.println(e.getMessage());
  }
  return products;
}
```

This method is the **core data-loading mechanism**. It reads the CSV line by line using `BufferedReader`, skips the header row, splits each line by comma, and parses every field into a `Product` object added to an `ArrayList`.

CODE SNIPPET #2

Inline Table Editing

```
tblProducts.getModel().addTableModelListener(e ->
{
  int row = e.getFirstRow();
  int col = e.getColumn();
  Object value = tblProducts.getModel()
                  .getValueAt(row, col);
  Product p = products.get(row);
  try {
    switch (col) {
      case 1: p.setName(value.toString()); break;
      case 2: p.setCategory(value.toString()); break;
      case 3: p.setBrand(value.toString()); break;
      case 4: p.setPrice(
          Double.parseDouble(value.toString())); break;
      case 5: p.setQuantity(
          Integer.parseInt(value.toString())); break;
      case 6: p.setSupplier(value.toString()); break;
    }
  } catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(null,
      "Invalid value!", "Error",
      JOptionPane.ERROR_MESSAGE);
    showProducts();
  }
});
```

A `TableModelListener` detects every cell edit and immediately updates the corresponding `Product` object in memory. If the user enters an invalid value — for example, text in the Price field — a validation error is displayed and the table is restored to its previous state.

CODE SNIPPET #3

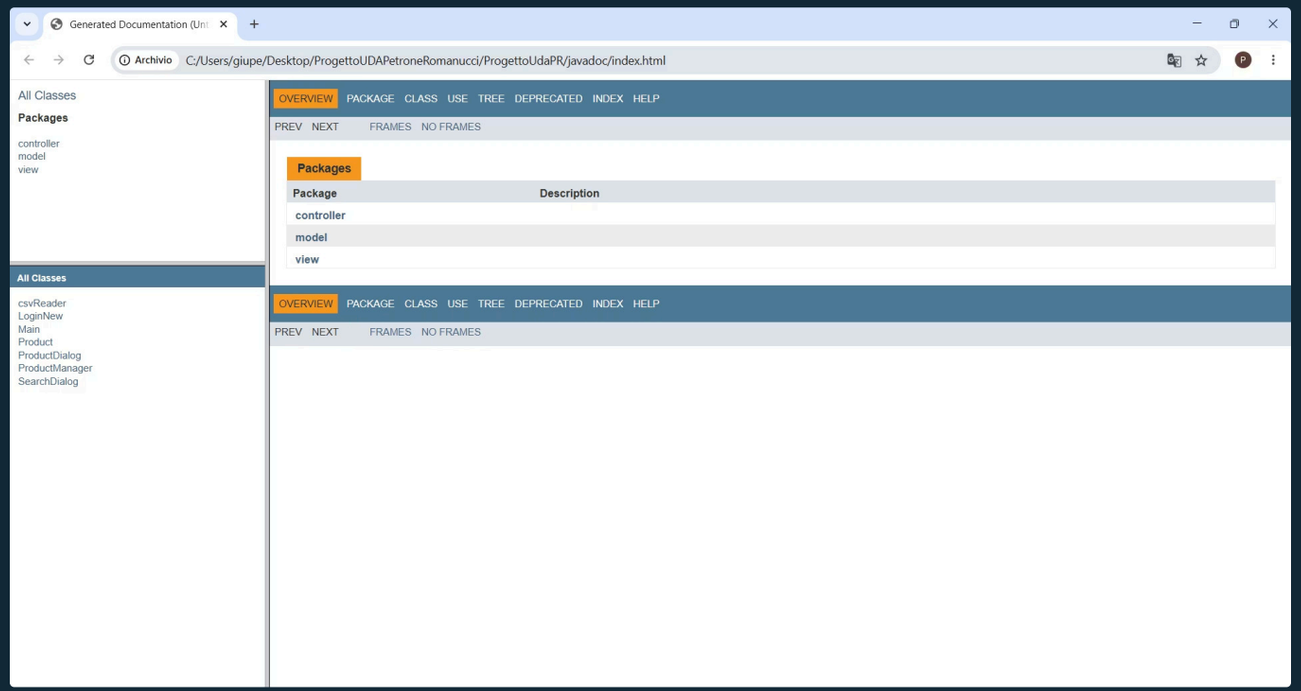
Form Validation

```
try {
  String name = txtName.getText().trim();
  String category = txtCategory.getText().trim();
  String brand = txtBrand.getText().trim();
  String supplier = txtSupplier.getText().trim();
  double price = Double.parseDouble(
                txtPrice.getText().trim());
  int quantity = Integer.parseInt(
                txtQuantity.getText().trim());
  if (name.isEmpty() || category.isEmpty() ||
     brand.isEmpty() || supplier.isEmpty()) {
    JOptionPane.showMessageDialog(this,
      "All fields are required!", "Error",
      JOptionPane.ERROR_MESSAGE);
    return;
  }
  if (price < 0 || quantity < 0) {
    JOptionPane.showMessageDialog(this,
      "Price and quantity must be positive!",
      "Error", JOptionPane.ERROR_MESSAGE);
    return;
  }
  product = new Product(0, name, category,
              brand, price, quantity, supplier);
  saved = true;
  dispose();
} catch (NumberFormatException e) {
  JOptionPane.showMessageDialog(this,
    "Price and quantity must be numbers!",
    "Error", JOptionPane.ERROR_MESSAGE);
}
```

Before creating a new `Product`, this method validates all form fields: it checks that text fields are not empty, that price and quantity are valid numbers, and that both are non-negative. Only when all checks pass is the dialog closed and the product saved.

DOCUMENTATION

Javadoc Documentation



Javadoc comments were written above each class and method using the `/** */` syntax. The HTML documentation was generated directly from NetBeans via **Run → Generate Javadoc**.

This practice improves code maintainability and makes the project easier to understand and navigate for other developers reviewing or extending the codebase.

Well-documented code is a hallmark of professional software engineering — it reduces onboarding time and supports long-term project sustainability.

CHALLENGES & FUTURE

Challenges Overcome & Roadmap Ahead

JTable Layout

The table didn't fill the window. **Solution:** Wrapped in `JScrollPane` and added via `BorderLayout.CENTER` for dynamic resizing.

ConcurrentModificationException

Thrown when editing cells during model rebuild. **Solution:** `showProducts()` rebuilds the `DefaultTableModel` and re-attaches the listener fresh each time.

Future Improvements

- Replace CSV with SQLite or MySQL for scalability
- Add multi-role authentication (admin, employee)
- Implement sales and invoice tracking
- Generate PDF reports for inventory and sales
- Add low-stock alerts and notifications
- Develop a web or mobile version